DocuSense

An AI-Powered Student Documentation Summarizer and Q&A Assistant

Final Semester Project Report

Submitted by: Demo Student

Department of Computer Applications, Demo College of Technology

Academic year 2026

Abstract

This project presents DocuSense, a local web application that helps teachers review long student project documents. A teacher uploads a PDF report and receives a structured summary with six sections: context, objectives, methodology, results, limitations and suggested viva questions. The teacher can then ask follow-up questions in a chat window. Every answer is produced only from passages retrieved from the uploaded document, using a technique called retrieval-augmented generation (RAG). When the document does not contain the answer, the system replies with a fixed sentence instead of guessing.

DocuSense is built in Python with LangChain, the Chroma vector database, the Ollama model runner and a Streamlit interface. It uses only free, open-source tools and runs entirely on the teacher's own computer, so no student document is sent to an outside service. On a test set of 13 questions the system answered all 9 answerable questions correctly and refused all 4 unanswerable ones. The project also documents its weaknesses honestly, including slow summary generation and the limits of a small language model.

1. Introduction

1.1 Background

In most colleges, students submit a final semester project report that can run from twenty to one hundred pages. Teachers must read each report before the viva voce examination, check whether the work meets the objectives, and prepare questions. When one teacher supervises or examines many students, this reading becomes the slowest part of the evaluation process.

1.2 Problem statement

Reading every report in full takes a long time, and the depth of review varies with how tired or rushed the reader is. Two teachers reading the same report may focus on different parts. Teachers also need to verify specific facts quickly, such as which dataset was used or what accuracy was reported, without searching through every page by hand.

1.3 Proposed solution

DocuSense gives each report a consistent first-pass review. It produces the same six-part summary for every document and lets the teacher ask direct questions with answers tied to page numbers. The tool is meant to support the teacher, not to replace the teacher's judgement, and every screen carries a notice that AI-generated content must be verified against the original document.

1.4 Motivation

The author completed a course on LangChain and wanted to build a practical project instead of a toy example. Document question answering was chosen because it solves a real problem in the author's own college, it can be built without paid services, and it teaches the full pipeline from PDF parsing to a finished interface.

1.5 Organisation of this report

Section 2 states the objectives and scope. Section 3 reviews the technology. Section 4 describes the system design. Section 5 explains the implementation. Section 6 covers the evaluation method, Section 7 gives the results, Section 8 discusses limitations and challenges, and Section 9 concludes with future work.

2. Objectives and Scope

2.1 Objectives

The main objectives of the project are listed below.

- To extract text from a student project PDF and keep the page number of every passage.
- To produce a structured summary with six fixed sections, including five suggested viva questions.
- To answer questions strictly from the uploaded document and cite the pages used.
- To refuse politely, with a fixed sentence, when the document does not support an answer.
- To run completely offline on an ordinary laptop with about 8 GB of RAM and no graphics card.
- To measure accuracy with a repeatable test set instead of relying on impressions.

2.2 Scope

The system accepts text-based PDF files. Each uploaded document is stored in its own collection so that answers about one student never mix with another student's work. The interface supports uploading, summarising, chatting and downloading the summary as a Markdown file. A light and a dark theme are provided.

2.3 Out of scope

Scanned PDFs without a text layer are not supported because optical character recognition (OCR) is not implemented. The system does not grade or score students, does not detect plagiarism, and has no user login. These items are listed as future work.

2.4 Target users

The primary users are teachers and project guides who review final semester documentation. A secondary use is a student who wants to check what questions a reviewer might ask about the report before the viva.

2.5 Success criteria

The project is considered successful if retrieval finds the right passage for nearly every answerable question, if unanswerable questions are always refused, if the full workflow works on a CPU-only laptop, and if the limitations are measured and reported honestly.

3. Technology Background

3.1 Retrieval-augmented generation

A language model alone can produce fluent answers that are wrong. Retrieval-augmented generation reduces this risk. The document is first split into small pieces called chunks. When a question arrives, the system finds the chunks most similar to the question and gives only those chunks to the model, with an instruction to answer from them alone. The model therefore works like a reader with the relevant pages open, not like a memory.

3.2 Embeddings and the vector database

An embedding is a list of numbers that captures the meaning of a piece of text. Texts with similar meaning have similar embeddings. DocuSense uses the nomic-embed-text model to create a 768-number embedding for every chunk and stores them in Chroma, a vector database that runs locally and saves its data to a folder on disk.

3.3 The language model and Ollama

Ollama is a free program that runs open-source language models on a personal computer. The project uses the llama3.2:3b model, which has about three billion parameters and needs roughly 2 GB of memory. It was chosen because the development laptop has about 8 GB of RAM and no usable graphics card. Larger models were rejected as too slow and too heavy.

3.4 LangChain and Streamlit

LangChain provides ready-made parts for text splitting, vector stores and model calls. Streamlit turns a Python script into a web page without separate front-end code. Together they allowed the whole application to be written in one language.

3.5 Why local processing matters

Student reports can contain personal names, institution details and unpublished ideas. Sending them to a cloud service raises privacy questions. Running every step locally avoids that problem, at the cost of speed and some answer quality, a trade-off that is discussed again in Section 8.

4. System Design

4.1 Architecture overview

The system has two pipelines that share one vector store. The ingestion pipeline prepares a document once. The summary pipeline and the question-answering pipeline then read from the same stored chunks.

4.2 Ingestion pipeline

- The PDF is read page by page with the pypdf library. Empty pages are skipped.
- Whitespace is cleaned while paragraph breaks are kept.
- Text is split into chunks of 1000 characters with an overlap of 150 characters, and each chunk keeps its page number.
- Each chunk is embedded and stored in its own Chroma collection with a unique document identifier.
- A scanned or empty PDF is detected and rejected with a clear message.

4.3 Summary pipeline

A long document cannot fit into one prompt of a small model, so the summary uses a map-reduce method. In the map step the chunks are grouped into up to ten parts, and the model writes short notes for each part. In the reduce step the model combines all notes into the final six-section summary. The finished summary is saved to disk, so the teacher waits only once per document.

4.4 Question-answering pipeline

The question is embedded, and the six most similar chunks are retrieved. A strict prompt tells the model to answer only from those chunks and to reply with the sentence "I could not find that in the document." if the answer is missing. The program then removes the model's own page citations and shows page numbers it has checked against the retrieved chunks. Numbers in the answer that do not appear in the retrieved text trigger a warning.

4.5 Design rules

- The model never answers from its own knowledge.
- Each student's document lives in a separate collection.
- Model names and sizes are kept in one configuration file so they can be swapped later.
- Every error shows a readable message instead of a crash.

5. Implementation

5.1 Project structure

The code is divided so that each file has one job. The file config.py holds model names, chunk sizes and retrieval settings. The file ingest.py turns a PDF into stored chunks. The file qa.py answers questions. The file summarize.py builds the summary. The file store.py keeps a registry of uploaded documents and a cache of saved summaries. The file ui.py holds the styling, and app.py is the Streamlit interface. A separate evaluate.py runs the tests described in Section 6.

5.2 Handling citations

Early versions let the model write its own page numbers. Testing showed that a small model sometimes invents a page that does not exist. The final version removes the model's citations and shows only pages that were really retrieved. When the model names no page, the interface labels the two best-matching pages as "Possible sources" so that the teacher is not misled.

5.3 The number check

A common failure of small models is quoting a wrong figure. DocuSense compares every number of two or more digits in an answer with the retrieved text. A number that is not found there is reported in a warning box. The check cannot catch a number that exists in the document but belongs to a different question.

5.4 Interface

The interface has a sidebar for choosing a document and uploading a new one, a top bar that shows whether Ollama is running, and two views. The Summary view shows six coloured cards. The Chat view keeps the input box at the bottom and shows an animated search indicator while an answer is being prepared. A dark mode switch changes the whole theme.

5.5 Safety features

- Uploaded file names are cleaned before they are saved.
- Duplicate uploads are detected with a file fingerprint and reused.
- The app listens only on the local address, not on the network.
- Student data folders are excluded from version control.

6. Evaluation Method

6.1 Test document

The system was tested on a 15-page research paper, which became 49 chunks of 1000 characters. A research paper was used because no real student reports could be shared. The same method can be repeated on any other document.

6.2 Question set

The test set has 13 questions. Nine are answerable and come with evidence keywords that must appear in the retrieved text. Four are unanswerable and include general knowledge questions and two traps that look answerable but are not, for example a request for a result on a language pair the paper never tested. A correct system must refuse all four.

6.3 Metrics

- Evidence retrieved: does the right passage appear among the top retrieved chunks?
- Answer correct: does an answer to an answerable question contain the expected keyword?
- Correct refusals: are unanswerable questions refused?
- False refusals: are answerable questions refused by mistake?
- Shown pages contain the evidence: do the displayed pages really hold the answer?
- Answers with unverified numbers: how often does the number check raise a warning?
- Time: average seconds per question and total time for a full summary.

6.4 Chunk-size experiment

To choose the chunk size, the document was indexed at four sizes: 500, 800, 1000 and 1500 characters. For each size the retrieval metric was measured, and the temporary indexes were deleted afterwards.

6.5 Test environment

All tests ran on a laptop with about 8 GB of RAM, no usable graphics card and Windows. The chat model was llama3.2:3b and the embedding model was nomic-embed-text. Each experiment was run through a script, so results can be reproduced.

7. Results

7.1 Question answering

Metric	Result
Evidence found in top 6 retrieved chunks	9 of 9
Answer correct on answerable questions	9 of 9
Correct refusals on unanswerable questions	4 of 4
False refusals on answerable questions	0 of 9
Shown pages contain the evidence	9 of 9
Answers with unverified numbers	0 of 9
Model named a page itself	7 of 9
Average time per question	about 31 seconds

7.2 Chunk-size comparison

Chunk size	Chunks	Questions with evidence in top 6
500 characters	96	8 of 9
800 characters	66	8 of 9
1000 characters	49	9 of 9
1500 characters	38	8 of 9

7.3 Summary generation

A full summary of the 15-page paper took about 11 minutes on the test laptop, because the model reads ten parts one after another and then writes the final text. The summary contained all six sections and the correct headline numbers. Because it is saved after the first run, later visits are instant.

7.4 Interpretation

A chunk size of 1000 characters scored best, but the other sizes missed only one question, and one question is too small a difference to call a real trend. The honest conclusion is that 1000 characters was not worse than the alternatives.

8. Limitations and Challenges

8.1 Limitations

- The test set is small: one document and 13 questions, with keywords chosen by the author after reading the paper. The results are a sanity check, not proof of accuracy.
- A three-billion-parameter model sometimes makes factual or formatting mistakes in summaries. Early summaries contained wrong attributions and a contradictory objectives section.
- Page citations are approximate. When the model names no page, the interface shows only the best-matching pages, labelled as possible sources.
- The number check misses figures that exist in the document but answer a different question, and it cannot detect contradictions inside the document.
- Scanned PDFs are rejected because OCR is not implemented.
- Summary generation is slow on a CPU, about ten minutes for a short document.
- There is no login and no delete button, so the app must not be exposed on a network.

8.2 Challenges faced

The first plan used a no-code workflow tool, but its free trial expired before the workflow could run, so the project was rebuilt in Python. The laptop's limited memory forced the choice of a small model. Small differences in how the model formatted its answers caused several bugs in citation handling, and one test question was graded as wrong only because the source paper contained two different values for the same score. Each of these problems was investigated, fixed in code where possible, and recorded.

8.3 Ethical considerations

An automatic summary can sound confident even when it is wrong. For this reason the interface always shows a warning that AI-generated content must be verified, and the documentation states that the tool supports a teacher but does not replace one.

9. Conclusion and Future Work

9.1 Conclusion

DocuSense shows that a useful document review assistant can be built with free tools on an ordinary laptop. It produces a structured six-part summary, answers questions from the uploaded document with page references, and refuses when the document has no answer. On the test set it answered all nine answerable questions correctly and refused all four unanswerable ones. The project also shows the value of measuring a system instead of trusting it: several early weaknesses, such as invented page numbers, were found only through testing.

9.2 Future work

- Add OCR so that scanned reports can be processed.
- Protect the system against prompt injection, where text inside a PDF tries to give the model new instructions.
- Add a delete button and a teacher login.
- Add hybrid search that combines keyword and meaning-based matching, and re-rank the results.
- Offer an optional stronger cloud model for better summaries, with a clear privacy warning.
- Build a larger test set from real student reports and several teachers' questions.
- Warn the user when some PDF pages contain no extractable text.

9.3 Learning outcomes

The project gave the author practical experience with PDF processing, text chunking, embeddings, vector databases, prompt design, automated evaluation, web interface design and honest reporting of limitations.

9.4 Tools used

Python 3.11, LangChain, Chroma, Ollama with llama3.2:3b and nomic-embed-text, pypdf, Streamlit and the standard unittest library.

9.5 Note on this document

This report was written as a demonstration document for testing DocuSense. The student name and college are fictional, and the evaluation figures describe the author's own test runs.